The \p construct gives you an encoding-aware way of matching a character with a particular Unicode property (shown in Table 11, Unicode character properties, on page 445):

```
# encoding: utf-8
string = "∂y/∂x = 2πx"
show_regexp(string, /\p{Alnum}/) # => ∂->y<-/∂x = 2πx
show_regexp(string, /\p{Digit}/) # => ∂y/∂x = ->2<-πx
show_regexp(string, /\p{Space}/) # => ∂y/∂x-> <-= 2πx
show_regexp(string, /\p{Greek}/) # => ∂y/∂x = 2->π<-x
show_regexp(string, /\p{Graph}/) # => ->∂<-y/∂x = 2πx
```

General Delimited Input

In addition to the normal quoting mechanism, Ruby supports a generalized delimiter sequence that allows you to write alternative forms of literal strings, arrays of strings and symbols, regular expressions, and shell commands. All these literals start with a percent character followed by a single character that identifies the literal's type. These characters are summarized in the following table; the actual literal values are described in the corresponding sections later in this chapter.

Type	Meaning	Example
%q	Single-quoted string	%q{\a and #{1+2} are literal}
%Q, %	Double-quoted string	%Q{\a and #{1+2} are expanded}
%w, %W	Array of strings	%w[one two three]
%i, %I	Array of symbols	%i[one two three]
%r	Regular expression pattern	%r{cat|dog}
%s	A symbol	%s!a symbol!
%x	Shell command	%x(df -h)

Unlike their lowercase counterparts, %I, %Q, and %W will preform interpolation:

```
%i{one digit#{1+1} three} # => [:one, :"digit\#{1+1}", :three]
%I{one digit#{1+1} three} # => [:one, :digit2, :three]
%q{one digit#{1+1} three} # => "one digit\#{1+1} three"
%Q{one digit#{1+1} three} # => "one digit2 three"
%w{one digit#{1+1} three} # => ["one", "digit\#{1+1}", "three"]
%W{one digit#{1+1} three} # => ["one", "digit2", "three"]
```

Following the type character is a delimiter, which can be any nonalphanumericic or non-multibyte character. If the delimiter is one of the characters (, [, {, or <, the literal consists of the characters up to the matching closing delimiter, taking account of nested delimiter pairs. For all other delimiters, the literal comprises the characters up to the next occurrence of the delimiter character.

```
%q/this is a string/
%q-string-
%q(a (nested) string)
```

Delimited strings may continue over multiple lines; the line endings and all spaces at the start of continuation lines will be included in the string:

```
meth = %q{def fred(a)
  a.each {|i| puts i }
```